

Laporan Soal Latihan UAS

Aplikasi Catatan Keuangan API Informasi Umum

Nama: Bilal

NIM: 241552010004

Tanggal Dikumpulkan: Minggu, 5 Juli 2026

Latar Belakang

Sebuah startup ingin membangun aplikasi Catatan Keuangan Pribadi berbasis web. Anda ditugaskan untuk membangun backend API-nya. Aplikasi ini memungkinkan pengguna mencatat setiap transaksi keuangan mereka (pemasukan dan pengeluaran), serta melihat ringkasan kondisi keuangan mereka.

Ketentuan Teknis

- 1) Framework: Express.js
- 2) Database: SQLite menggunakan Prisma ORM
- 3) Autentikasi: JWT (JSON Web Token)
- 4) Hash password: bcryptjs
- 5) File `.env`, `package.json`, `db.js`, `index.js`, dan `prisma/schema.prisma` sudah disediakan sebagai starter
- 6) Jalankan `npm run prisma migrate dev --name init` setelah schema siap

Struktur File yang Harus Dibuat

Latihan-uas-NIM-NAMA/

— middleware/	
— authGuard.js	← middleware cek JWT token
— adminOnly.js	← middleware cek role admin
— routes/	
— auth.js	← endpoint register & login
— transaksi.js	← endpoint CRUD transaksi

Teknis Pengerjaan

SOAL 1 — Middleware Autentikasi

Lokasi File: *middleware/authGuard.js*

```
C:\Users\ASUS>wsl.exe
fufufafa@whoiam: /mnt/c/Users/ASUS$ cd ~ && source 0-0/bin/activate && cd /home/fufufafa/241552010004-Web-Application/W15-latihan-uas/
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ cat middleware/authGuard.js
const jwt = require("jsonwebtoken")
const prisma = require("../db")

module.exports = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization

    if (!authHeader || !authHeader.startsWith("Bearer ")) {
      return res.status(401).json({
        message: "Token tidak ditemukan"
      })
    }

    const token = authHeader.split(" ")[1]

    let decoded

    try {
      decoded = jwt.verify(token, process.env.JWT_SECRET)
    } catch (err) {
      return res.status(401).json({
        message: "Token tidak valid"
      })
    }

    const user = await prisma.user.findUnique({
      where: {
        id: decoded.userId
      },
      select: {
        id: true,
        email: true,
        nama: true,
        role: true
      }
    })

    if (!user) {
      return res.status(401).json({
        message: "User tidak ditemukan"
      })
    }

    req.user = user

    next()
  } catch (err) {
    next(err)
  }
}
```

Deskripsi: Pada bagian ini saya membuat middleware *authGuard.js* yang berfungsi sebagai pengaman sebelum pengguna dapat mengakses endpoint yang membutuhkan autentikasi. Middleware ini terlebih dahulu memeriksa apakah request memiliki header *Authorization* dengan format *Bearer Token*. Jika token tidak ada atau formatnya tidak sesuai, sistem akan langsung mengembalikan pesan bahwa pengguna belum memiliki akses. Selanjutnya token akan diverifikasi menggunakan *JWT Secret* untuk memastikan token benar-benar valid. Setelah berhasil diverifikasi, sistem mencari data pengguna di database berdasarkan *userId* yang terdapat di dalam token. Apabila data pengguna ditemukan, informasi penting seperti *id*, *email*, *nama*, dan *role* disimpan ke dalam *req.user* agar dapat digunakan pada proses berikutnya tanpa perlu mengambil data ulang. Dengan cara ini, proses autentikasi menjadi lebih aman sekaligus memudahkan setiap endpoint dalam mengetahui identitas pengguna yang sedang login.

SOAL 2 — Middleware Admin

Lokasi File: *middleware/adminOnly.js*

```
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ cat middleware/adminOnly.js
const adminOnly = (req, res, next) => {
  if (!req.user) {
    return res.status(401).json({
      message: "Autentikasi diperlukan"
    })
  }

  if (req.user.role !== "admin") {
    return res.status(403).json({
      message: "Akses ditolak. Hanya admin yang diizinkan."
    })
  }

  next()
}

module.exports = adminOnly
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ |
```

Deskripsi: Pada file *adminOnly.js*, saya membuat sebuah middleware yang berfungsi untuk membatasi akses ke halaman atau endpoint tertentu agar hanya dapat digunakan oleh pengguna yang memiliki peran sebagai admin. Middleware ini terlebih dahulu memastikan bahwa pengguna sudah berhasil melakukan autentikasi melalui middleware sebelumnya dengan mengecek apakah data *req.user* tersedia. Jika pengguna belum login, sistem akan mengembalikan respons 401 Unauthorized. Selanjutnya, middleware memeriksa nilai *role* pada data pengguna. Jika *role* bukan *admin*, maka sistem akan menolak akses dan mengembalikan respons 403 Forbidden. Apabila semua pengecekan berhasil, middleware akan menjalankan *next()* sehingga proses dapat dilanjutkan ke endpoint berikutnya. Menurut saya, pemisahan proses autentikasi dan otorisasi seperti ini membuat alur program menjadi lebih rapi, mudah dipahami, dan memudahkan ketika middleware tersebut ingin digunakan kembali pada endpoint lain yang membutuhkan hak akses khusus.

SOAL 3 — Route Autentikasi (routes/auth.js)

Lokasi File: *routes/auth.js*

```
(0-0) fufufafa@whoian:~/241552010004-Web-Application/W15-latihan-uas$ cat routes/auth.js
const express = require("express")
const bcrypt = require("bcryptjs")
const jwt = require("jsonwebtoken")
const prisma = require("../db")
const router = express.Router()

router.post("/register", async (req, res, next) => {
  try {
    const { email, nama, password } = req.body
    if (!email || !nama || !password) {
      return res.status(400).json({
        message: "Email, nama, dan password wajib diisi"
      })
    }
    if (password.length < 8) {
      return res.status(400).json({
        message: "Password minimal 8 karakter"
      })
    }
    const existingUser = await prisma.user.findUnique({
      where: {
        email
      }
    })
    if (existingUser) {
      return res.status(409).json({
        message: "Email sudah terdaftar"
      })
    }
    const hashedPassword = await bcrypt.hash(password, 10)
    const user = await prisma.user.create({
      data: {
        email,
        nama,
        password: hashedPassword,
        role: "user"
      }
    })
    const { password: _, ...safeUser } = user
    res.status(201).json({
      message: "Registrasi berhasil",
      user: safeUser
    })
  } catch (err) {
    next(err)
  }
})

router.post("/login", async (req, res, next) => {
  try {
    const { email, password } = req.body
    if (!email || !password) {
      return res.status(400).json({
        message: "Email dan password wajib diisi"
      })
    }
    const user = await prisma.user.findUnique({
      where: {
        email
      }
    })
    if (!user) {
      return res.status(401).json({
        message: "Email atau password salah"
      })
    }
    const validPassword = await bcrypt.compare(
      password,
      user.password
    )
    if (!validPassword) {
      return res.status(401).json({
        message: "Email atau password salah"
      })
    }
    const token = jwt.sign(
      {
        userId: user.id,
        email: user.email,
        nama: user.nama,
        role: user.role
      },
      process.env.JWT_SECRET,
      {
        expiresIn: "7d"
      }
    )
    const { password: _, ...safeUser } = user
    res.json({
      message: "Login berhasil",
      token,
      user: safeUser
    })
  } catch (err) {
    next(err)
  }
})

module.exports = router
(0-0) fufufafa@whoian:~/241552010004-Web-Application/W15-latihan-uas$
```

Deskripsi: Pada file *routes/auth.js*, saya membuat dua endpoint utama yaitu *register* dan *login* sebagai dasar proses autentikasi pengguna pada aplikasi. Pada endpoint *register*, saya menambahkan validasi agar email, nama, dan password wajib diisi, serta memastikan password memiliki minimal 8 karakter. Saya juga melakukan pengecekan apakah email sudah pernah digunakan sebelumnya untuk menghindari data ganda. Sebelum data disimpan ke database menggunakan Prisma ORM, password diubah menjadi bentuk hash menggunakan *bcryptjs* agar lebih aman. Sementara itu, pada endpoint *login*, saya melakukan validasi data yang dikirim pengguna, kemudian mencocokkan password yang dimasukkan dengan password yang telah di-hash di database. Jika proses autentikasi berhasil, sistem akan membuat JSON Web Token (JWT) yang berisi informasi penting pengguna seperti ID, email, nama, dan role dengan masa berlaku selama tujuh hari. Selain itu, saya memastikan bahwa password tidak pernah dikirim kembali ke client sebagai bagian dari respons, karena menurut saya menjaga keamanan data pengguna merupakan salah satu hal penting dalam pengembangan aplikasi backend.

SOAL 4 — Route Transaksi

Lokasi File: *routes/transaksi.js*

```
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ cat routes/transaksi.js
const express = require("express")
const prisma = require("../db")
const authGuard = require("../middleware/authGuard")
const router = express.Router()
router.use(authGuard)
const isAdmin = (req) => req.user.role === "admin"
const canAccess = (req, transaksi) => {
  return isAdmin(req) || transaksi.userId === req.user.id
}

router.post("/", async (req, res, next) => {
  try {
    const { judul, jumlah, jenis, kategori, tanggal } = req.body

    if (!judul || jumlah === undefined || !jenis || !kategori) {
      return res.status(400).json({
        message: "Judul, jumlah, jenis, dan kategori wajib diisi"
      })
    }

    if (!["pemasukan", "pengeluaran"].includes(jenis)) {
      return res.status(400).json({
        message: "Jenis harus pemasukan atau pengeluaran"
      })
    }

    if (isNaN(jumlah) || Number(jumlah) <= 0) {
      return res.status(400).json({
        message: "Jumlah harus berupa angka positif"
      })
    }

    const transaksi = await prisma.transaksi.create({
      data: {
        judul,
        jumlah: Number(jumlah),
        jenis,
        kategori,
        tanggal: tanggal ? new Date(tanggal) : new Date(),
        userId: req.user.id
      }
    })

    res.status(201).json({
      message: "Transaksi berhasil ditambahkan",
      transaksi
    })
  } catch (err) {
    next(err)
  }
})
```



```

router.get("/", async (req, res, next) => {
  try {
    const transaksi = await prisma.transaksi.findMany({
      where: isAdmin(req) ? {} : { userId: req.user.id },
      include: {
        user: {
          select: {
            id: true,
            nama: true,
            email: true
          }
        }
      },
      orderBy: {
        tanggal: "desc"
      }
    })

    res.json(transaksi)
  } catch (err) {
    next(err)
  }
})

router.get("/ringkasan", async (req, res, next) => {
  try {
    const transaksi = await prisma.transaksi.findMany({
      where: isAdmin(req) ? {} : { userId: req.user.id }
    })

    const totalPemasukan = transaksi
      .filter(t => t.jenis === "pemasukan")
      .reduce((sum, t) => sum + t.jumlah, 0)

    const totalPengeluaran = transaksi
      .filter(t => t.jenis === "pengeluaran")
      .reduce((sum, t) => sum + t.jumlah, 0)

    res.json({
      totalPemasukan,
      totalPengeluaran,
      saldo: totalPemasukan - totalPengeluaran
    })
  } catch (err) {
    next(err)
  }
})

```

```

router.get("/:id", async (req, res, next) => {
  try {
    const transaksi = await prisma.transaksi.findUnique({
      where: {
        id: Number(req.params.id)
      },
      include: {
        user: {
          select: {
            id: true,
            nama: true,
            email: true
          }
        }
      }
    })
  } catch (err) {
    next(err)
  }

  if (!transaksi) {
    return res.status(404).json({
      message: "Transaksi tidak ditemukan"
    })
  }

  if (!canAccess(req, transaksi)) {
    return res.status(403).json({
      message: "Akses ditolak"
    })
  }

  res.json(transaksi)
} catch (err) {
  next(err)
}
})

```

```
router.put("/:id", async (req, res, next) => {
  try {

    const transaksi = await prisma.transaksi.findUnique({
      where: {
        id: Number(req.params.id)
      }
    })

    if (!transaksi) {
      return res.status(404).json({
        message: "Transaksi tidak ditemukan"
      })
    }

    if (!canAccess(req, transaksi)) {
      return res.status(403).json({
        message: "Akses ditolak"
      })
    }

    const data = {}

    if (req.body.judul !== undefined)
      data.judul = req.body.judul

    if (req.body.kategori !== undefined)
      data.kategori = req.body.kategori

    if (req.body.tanggal !== undefined)
      data.tanggal = new Date(req.body.tanggal)

    if (req.body.jenis !== undefined) {
      if (!["pemasukan", "pengeluaran"].includes(req.body.jenis)) {
        return res.status(400).json({
          message: "Jenis harus pemasukan atau pengeluaran"
        })
      }
      data.jenis = req.body.jenis
    }

    if (req.body.jumlah !== undefined) {
      if (isNaN(req.body.jumlah) || Number(req.body.jumlah) <= 0) {
        return res.status(400).json({
          message: "Jumlah harus berupa angka positif"
        })
      }

      data.jumlah = Number(req.body.jumlah)
    }

    const updated = await prisma.transaksi.update({
      where: {
        id: transaksi.id
      },
      data
    })

    res.json({
      message: "Transaksi berhasil diupdate",
      transaksi: updated
    })

  } catch (err) {
    next(err)
  }
})
```

```

router.delete("/:id", async (req, res, next) => {
  try {
    const transaksi = await prisma.transaksi.findUnique({
      where: {
        id: Number(req.params.id)
      }
    })

    if (!transaksi) {
      return res.status(404).json({
        message: "Transaksi tidak ditemukan"
      })
    }

    if (!canAccess(req, transaksi)) {
      return res.status(403).json({
        message: "Akses ditolak"
      })
    }

    await prisma.transaksi.delete({
      where: {
        id: transaksi.id
      }
    })

    res.json({
      message: "Transaksi berhasil dihapus"
    })

  } catch (err) {
    next(err)
  }
})

module.exports = router
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ |

```

Deskripsi: Pada file routes/transaksi.js, saya membuat seluruh endpoint yang berkaitan dengan pengelolaan transaksi keuangan, mulai dari menambahkan, melihat, memperbarui, menghapus, hingga menampilkan ringkasan keuangan. Saya berusaha membuat alur program yang mudah dipahami dengan menambahkan validasi pada setiap data yang masuk agar tidak terjadi kesalahan, misalnya memastikan jumlah transaksi bernilai positif dan jenis transaksi hanya berupa pemasukan atau pengeluaran. Selain itu, saya juga menerapkan pembatasan hak akses sehingga pengguna biasa hanya dapat mengelola transaksi miliknya sendiri, sedangkan admin dapat mengakses seluruh data.

FILE PENDUKUNG

```

(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ tree
.
├── db.js
├── index.js
├── middleware
│   ├── adminOnly.js
│   └── authGuard.js
├── package.json
├── prisma
│   └── schema.prisma
├── routes
│   ├── auth.js
│   └── transaksi.js
└──
4 directories, 8 files

```


db.js

```
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ cat db.js
const { PrismaClient } = require('@prisma/client')
const prisma = new PrismaClient()
module.exports = prisma
```

Deskripsi: File db.js merupakan file bawaan yang berfungsi sebagai penghubung antara aplikasi dengan database melalui Prisma Client. File ini membuat satu instance PrismaClient yang kemudian diekspor agar dapat digunakan kembali oleh seluruh route atau middleware yang membutuhkan akses ke database. Dengan cara ini, koneksi database menjadi lebih terpusat, rapi, dan mudah dikelola selama proses pengembangan aplikasi.

index.js

```
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ cat index.js
const express = require("express")
const dotenv = require("dotenv")

dotenv.config()

const authRoutes = require("./routes/auth")
const transaksiRoutes = require("./routes/transaksi")

const app = express()

app.use(express.json())

app.use("/api/auth", authRoutes)
app.use("/api/transaksi", transaksiRoutes)

app.use((err, req, res, next) => {
  console.error(err)

  res.status(err.status || 500).json({
    message: err.message || "Internal Server Error"
  })
})

const PORT = process.env.PORT || 3000

app.listen(PORT, () => {
  console.log(`Server berjalan di port ${PORT}`)
})
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ |
```

Deskripsi: File index.js merupakan file utama yang telah dimodifikasi dari starter project sebagai pusat konfigurasi aplikasi Express. Pada tahap ini dilakukan penambahan import route autentikasi dan transaksi, kemudian kedua route tersebut didaftarkan agar endpoint dapat diakses melalui URL yang sesuai. Selain itu, global error handler juga disesuaikan agar dapat mengembalikan kode status HTTP sesuai error yang terjadi, sehingga proses penanganan kesalahan menjadi lebih baik dan aplikasi dapat memberikan respons yang lebih informatif kepada pengguna.

package.json

```
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ cat package.json
{
  "name": "catatan-keuangan-api",
  "version": "1.0.0",
  "description": "REST API Catatan Keuangan Pribadi - Latihan UAS",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
  },
  "dependencies": {
    "@prisma/client": "^6.0.0",
    "bcryptjs": "^2.4.3",
    "dotenv": "^16.0.0",
    "express": "^4.18.0",
    "jsonwebtoken": "^9.0.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.0",
    "prisma": "^6.0.0"
  }
}
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ |
```

Deskripsi: File package.json merupakan file konfigurasi utama pada proyek Node.js yang sudah disediakan sebagai bagian dari starter project. File ini berisi informasi mengenai proyek, daftar dependency yang digunakan seperti Express, Prisma, JWT, dan bcryptjs, serta script untuk menjalankan aplikasi dalam mode normal maupun development. Dengan adanya file ini, proses instalasi dependency menjadi lebih mudah karena seluruh kebutuhan proyek sudah didefinisikan di dalamnya.

prisma/schema.prisma

```
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ cat prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model User {
  id          Int           @id @default(autoincrement())
  email       String        @unique
  nama        String
  password    String
  role        String        @default("user")
  createdAt   DateTime       @default(now())
  updatedAt   DateTime       @updatedAt
  transaksi   Transaksi[]
}

model Transaksi {
  id          Int           @id @default(autoincrement())
  judul       String
  jumlah      Float
  jenis       String
  kategori    String
  tanggal    DateTime       @default(now())
  createdAt   DateTime       @default(now())
  updatedAt   DateTime       @updatedAt
  userId      Int
  user        User          @relation(fields: [userId], references: [id])
}
```

Deskripsi: File `schema.prisma` merupakan file bawaan yang digunakan Prisma ORM untuk mendefinisikan struktur database aplikasi. Pada proyek ini telah disediakan model `User` dan `Transaksi` beserta relasi di antara keduanya sehingga mahasiswa dapat langsung melakukan migrasi database tanpa harus merancang skema dari awal. Keberadaan file ini mempermudah proses pengembangan karena struktur database sudah sesuai dengan kebutuhan sistem yang akan dibangun.

Membuat `.env` dan `.gitignore`

```
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ cat .env
DATABASE_URL="file:./prisma/dev.db"

JWT_SECRET=super-secret-jwt-key

PORT=3000
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ cat .gitignore
node_modules/
.env
prisma/dev.db
prisma/dev.db-journal
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ |
```

Deskripsi: File `.gitignore` merupakan file bawaan yang berfungsi untuk menentukan file atau folder yang tidak perlu dimasukkan ke dalam repository Git. Pada proyek ini, file tersebut digunakan untuk mengabaikan folder `node_modules`, file `.env`, serta database SQLite yang bersifat lokal sehingga data sensitif dan file hasil generate tidak ikut terunggah. Dengan adanya file ini, repository menjadi lebih bersih, aman, dan hanya berisi source code yang benar-benar diperlukan. File `.env` digunakan untuk menyimpan konfigurasi aplikasi yang bersifat dinamis, seperti lokasi database SQLite, JWT Secret untuk proses autentikasi, serta nomor port yang digunakan server. Penggunaan file ini membuat konfigurasi tidak perlu dituliskan langsung pada source code sehingga lebih aman dan memudahkan ketika aplikasi dijalankan pada lingkungan yang berbeda.

Setup & Menjalankan Hasil

Install dependencies : npm install
Generate Prisma Client : npx prisma generate
Jalankan migrasi database : npx prisma migrate dev --name init
Jalankan server : npm run dev

```
C:\Users\ASUS>wsl.exe
fufufafa@whoiam: /mnt/c/Users/ASUS$
fufufafa@whoiam: /mnt/c/Users/ASUS$ cd ~ && source 0-0/bin/activate && cd ~/241552010004-Web-Application/W15-latihan-uas && npm install
added 145 packages, and audited 146 packages in 1m
25 packages are looking for funding
run 'npm fund' for details
found 0 vulnerabilities
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ npx prisma generate
Environment variables loaded from .env
Prisma schema loaded from prisma/schema.prisma
Update available 6.19.3 -> 7.8.0
This is a major update - please follow the guide at
https://pris.ly/d/major-version-upgrade
Run the following to update
npm i --save-dev prisma@latest
npm i @prisma/client@latest
✓Generated Prisma Client (v6.19.3) to ./node_modules/@prisma/client in 180ms
Start by importing your Prisma Client (See: https://pris.ly/d/importing-client)
Tip: Want to turn off tips and other hints? https://pris.ly/tip-4-nohints
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ npx prisma migrate dev --name init
Environment variables loaded from .env
Prisma schema loaded from prisma/schema.prisma
Datasource "db": SQLite database "dev.db" at "file:./prisma/dev.db"
SQLite database dev.db created at file:./prisma/dev.db
Applying migration '20260704163214_init'
The following migration(s) have been created and applied from new schema changes:
prisma/migrations/
├─ 20260704163214_init/
└─ migration.sql
Your database is now in sync with your schema.
✓Generated Prisma Client (v6.19.3) to ./node_modules/@prisma/client in 184ms
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ npm run dev
> catatan-keuangan-api@1.0.0 dev
> nodemon index.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter 'rs'
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting 'node index.js'
Server berjalan di port 3000
```

Testing Response

Note: Kontrak API Base URL <http://localhost:3000> dan endpoint yang membutuhkan login harus menyertakan header: *Authorization: Bearer <token>*

Auth Endpoints

'POST /api/auth/register'

```
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ curl -X POST http://localhost:3000/api/auth/register -H "Content-Type: application/json" -d '{"email": "budi@email.com", "nama": "Budi Santoso", "password": "Rahasia123"}'
{"message": "Registrasi berhasil", "user": {"id": 1, "email": "budi@email.com", "nama": "Budi Santoso", "role": "user", "createdAt": "2026-07-04T21:23:23.914Z", "updatedAt": "2026-07-04T21:23:23.914Z"}}(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ curl -X POST http://localhost:3000/api/auth/register -H "Content-Type: application/json" -d '{"email": "budi@email.com", "nama": "Budi Santoso", "password": ""}'
{"message": "Email, nama, dan password wajib diisi!"}(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ curl -X POST http://localhost:3000/api/auth/register -H "Content-Type: application/json" -d '{"email": "budi@email.com", "nama": "Budi Santoso", "password": "123"}'
{"message": "Password minimal 8 karakter!"}(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ curl -X POST http://localhost:3000/api/auth/register -H "Content-Type: application/json" -d '{"email": "budi@email.com", "nama": "Budi Santoso", "password": "Rahasia123"}'
{"message": "Email sudah terdaftar!"}(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ curl -X POST http://localhost:3000/api/auth/register -H "Content-Type: application/json" -d '{"email": "jamil@email.com", "nama": "Jamil", "password": "Rahasia123"}'
{"message": "Registrasi berhasil", "user": {"id": 2, "email": "jamil@email.com", "nama": "Jamil", "role": "user", "createdAt": "2026-07-04T21:25:17.968Z", "updatedAt": "2026-07-04T21:25:17.968Z"}}(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ curl -X POST http://localhost:3000/api/auth/register -H "Content-Type: application/json" -d '{"email": "admin@email.com", "nama": "Admin", "password": "Rahasia123"}'
{"message": "Registrasi berhasil", "user": {"id": 3, "email": "admin@email.com", "nama": "Admin", "role": "user", "createdAt": "2026-07-04T21:25:45.431Z", "updatedAt": "2026-07-04T21:25:45.431Z"}}(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$
```

Membuat 3 akun: 2 untuk user dengan nama Jamil dan Budi Santoso juga admin dengan nama admin. Mengecek response error pada validasi register.

Jamal berhasil update data miliknya sendiri, namun ketika melakukan update untuk angka negative jamal mendapatkan response “Jumlah harus berupa angka positif”, begitupun dengan response “Jenis harus pemasukan atau pengeluaran”, kemudian budi mencoba update data milik jamal namun gagal dengan response “Akses ditolak”, berbeda dengan admin yang bisa mengubah data milik user, yang terakhir Adalah memastikan error yang tampil karena id tidak valid.

'DELETE /api/transaksi/:id'

```
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$  
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ curl -X DELETE http://localhost:3000/api/transaksi/999 -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2V5SWQ1OjE3IiwiaWF0IjoiYWRtaW5hbnV29IiwibmFtYSI6ImFkbWU1Iiwicm9sZSI6ImFkbWU1IiwiaWF0IjoxNzgzMjA0MTk0LCJleHAiOjE3ODM4MDg5OTR9.4YR-Nc1aStZbYhE3nus12Fv0qBEWUS1Mb7VITVH0hg"  
[message: "Transaksi tidak ditemukan"] (0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$  
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ curl -X DELETE http://localhost:3000/api/transaksi/1 -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2V5SWQ1OjE3IiwiaWF0IjoiYWRtaW5hbnV29IiwibmFtYSI6ImFkbWU1Iiwicm9sZSI6ImFkbWU1IiwiaWF0IjoxNzgzMjA0MTk0LCJleHAiOjE3ODM4MDg5OTR9.4YR-Nc1aStZbYhE3nus12Fv0qBEWUS1Mb7VITVH0hg"  
[message: "Akses ditolak"] (0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$  
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ curl -X DELETE http://localhost:3000/api/transaksi/1 -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2V5SWQ1OjE3IiwiaWF0IjoiYWRtaW5hbnV29IiwibmFtYSI6ImFkbWU1Iiwicm9sZSI6ImFkbWU1IiwiaWF0IjoxNzgzMjA0MTk0LCJleHAiOjE3ODM4MDg5OTR9.4YR-Nc1aStZbYhE3nus12Fv0qBEWUS1Mb7VITVH0hg"  
[message: "Transaksi berhasil dihapus"] (0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$  
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ curl -X DELETE http://localhost:3000/api/transaksi/2 -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2V5SWQ1OjE3IiwiaWF0IjoiYWRtaW5hbnV29IiwibmFtYSI6ImFkbWU1Iiwicm9sZSI6ImFkbWU1IiwiaWF0IjoxNzgzMjA0MTk0LCJleHAiOjE3ODM4MDg5OTR9.4YR-Nc1aStZbYhE3nus12Fv0qBEWUS1Mb7VITVH0hg"  
[message: "Transaksi berhasil dihapus"] (0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$  
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$
```

Baik admin ataupun user biasa tidak bisa menghapus data dengan id yang tidak valid sebagaimana pada gambar, lalu Budi mencoba menghapus data milik jamal namun gagal dengan response “Akses ditolak”, dan ketika Jamal menghapus data miliknya hasilnya berhasil, dan yang terakhir adalah admin yang bisa menghapus data milik budi.

Database Schema (sudah disediakan)

```
(0-0) fufufafa@whoiam:~/241552010004-Web-Application/W15-latihan-uas$ cat prisma/schema.prisma  
generator client {  
  provider = "prisma-client-js"  
}  
  
datasource db {  
  provider = "sqlite"  
  url      = env("DATABASE_URL")  
}  
  
model User {  
  id          Int           @id @default(autoincrement())  
  email       String        @unique  
  nama        String  
  password    String  
  role        String        @default("user")  
  createdAt   DateTime      @default(now())  
  updatedAt   DateTime      @updatedAt  
  transaksi   Transaksi[]  
}  
  
model Transaksi {  
  id          Int           @id @default(autoincrement())  
  judul       String  
  jumlah     Float  
  jenis       String  
  kategori    String  
  tanggal    DateTime      @default(now())  
  createdAt   DateTime      @default(now())  
  updatedAt   DateTime      @updatedAt  
  userId      Int  
  user        User          @relation(fields: [userId], references: [id])  
}
```

PERNYATAAN INTEGRITAS AKADEMIK

Dengan ini saya menyatakan bahwa seluruh kode program di atas merupakan hasil pekerjaan mandiri saya sesuai aturan yang berlaku tanpa melakukan plagiarisme terhadap pekerjaan orang lain.

Hormat Saya, *Bilal*